

Agentium: A Governable Agent Operating System Immutable Run Contracts, Policy Gates, and Auditable Automation

Guimin Chen* Jiezhen Zhao†

Abstract

Enterprise LLM agents amplify risk when undisclosed tools, cross-tenant data paths, outbound sensitive payloads, or background workflows operate without attributable, versioned supervision. We present *Agentium*, an agent infrastructure that treats these concerns as explicit *control-plane obligations*: layered governance, immutable *run manifests*, joint policy adjudication (**PolicyEngine**), human gates (**ApprovalGate**), and auditable linkage keyed by stable run and tool identifiers, with proactive background automation routed through the same perimeter. Substitutable persistence, memory, schedulers, and learning adapters attach only under manifests, tenancy, and joinable lineage. Evidence comes from a reproducible artefact: nine tagged tests jointly satisfying acceptance hypotheses H1–H6, three governance profiles (**none/weak/full**), supplemental release gates, and a scripted three-way ablation (full / no-manifest / permissive) with a deterministic microbenchmark—no proprietary models or prompts (Section 10.2). Versioned ingress contracts (**message_disposition**, MCP execution tier, optional grounding hooks, and staged DLP semantics) bind behaviour at boundaries; DLP staging completeness across every integration is not asserted. Contributions are architectural and protocol-facing, backed by artefacts and scoped empirical evidence; we make no headline benchmark-superiority claims.

Keywords: LLM agents, control plane, governance policy enforcement, observability, auditability, human-in-the-loop, composable backends, reproducibility, run manifest, software architecture.

1 Introduction

ReAct-style tool loops [31] and recent production-style agent frameworks [29] have made tool-mediated autonomy common in research and early deployments, but operational risk is rarely reducible to a single model prompt: tool

*Email: chenguimin@foxmail.com. Implementation artefact: <https://github.com/GuiminChen/agentium>.

†Email: zhaojiezhen@foxmail.com.

invocations, channel integrations, and background jobs induce side effects that must align with declared policy, support incident review, and remain replayable. Mitigations confined to prompt hardening or coarse role-based access control alone leave systematic gaps between *stated intent*, *runtime behaviour*, and *post-hoc evidence* [9, 22].

Agentium addresses this gap with explicit *planes* for governance, coordination, control, execution, and resources. Cross-boundary effects pass through instrumented APIs with stable identifiers (`run_id`, `tool_use_id`, trace fields) instead of unconstrained model output [2, 3]. Section 3 thereafter states tenant assets, adversary classes entertained, explicit exclusions, and illustrative misuse compositions; those boundaries anchor the automatable gates summarised in Section 9 and the hypothesis-to-test correspondence in Appendix A.

The manuscript advances six *jointly held* claims—layered control plane with immutable run contracts; co-designed policy, human gates, and audit; governed background automation; a dark-path matrix with automatable acceptance criteria; governed extension points; and versioned ingress contracts with grounding hooks—each stated precisely and with its scope limits in the enumeration below.

Contributions.

1. **Layered control plane and immutable runs.** We scope enterprise risk as a control-plane problem with explicit threat-model boundaries and `run_manifest` snapshots that pin tool sets, policy versions, and routing profiles. *Layering* separates governance, coordination, the governed background plane, control, execution, and resources: tool calls, egress, and other externally consequential effects traverse instrumented ingress with stable identifiers (`run_id`, `tool_use_id`)—we do not pin compliance and accountability on a single undifferentiated model-tool loop or on prompts alone. *Immutable runs* mean that each accepted run is, for evidence purposes, bound to that snapshot (`declared_tools`, `policy_bundle_ref`, etc.): undeclared tools fail closed at ingress and registry alignment; policy references can be reconciled with active releases with joinable audit records, so incident review proceeds from the same run contract plus linked audit telemetry rather than retrofitting intent from unconstrained model text. Operational sketches of ingress, alignment, approvals, and invariants appear in Section 6.
2. **Policy, human gates, audit.** `PolicyEngine` adjudicates `allow`, human-in-the-loop requirements, and `deny` per tenant, run, and risk class so automated decisions are versioned and replayable rather than inferred from model tone alone. `ApprovalGate` suspends externally consequential actions at `tool_use_id` granularity when policy demands

human intervention, then resumes or terminates with explicit operator decisions—unconstrained natural language must not stand in for authorization. **Audit** binds each tool attempt and adjudication outcome to joinable fields (`run_id`, `tool_use_id`, policy and manifest references) for compliance evidence and incident replay; the three are co-designed as one enforcement chain, not three unrelated features.

3. **Governed background plane.** Interval and event-driven *triggers* with a `BackgroundDaemon` constitute an automation outer loop parallel to interactive sessions, yet remain subject to the same `PolicyEngine`, manifest, and tenancy perimeter as foreground work: `BackgroundPolicyGuard`-class checks prevent background execution from registering undeclared tools, enlarging egress surfaces, or circumventing human gates required for equivalent risk in interactive settings—avoiding *divergent enforcement* in which interactive channels are tightly controlled but scheduled automation is permissive. The plane inherits rate limits, circuit breaking, kill-switch semantics, bounded safe degradation of externally visible effects, and operator-mediated recovery paths (as in hypothesis H3), so that erroneous or noisy triggers remain bounded by deployment parameters and audit-backed controls rather than by unconstrained model-mediated re-entry.
4. **Dark-path matrix and acceptance protocol.** *Dark paths* denote off-path failure and abuse compositions frequently cited in operational security arguments: undeclared tools (shadow capabilities), outbound sensitive payloads (prompt constraints being insufficient), cross-tenant confusion, background and event-driven amplification absent proportional oversight, bypass of high-risk approvals, and brittle or unsafe recovery. The *matrix* associates each class with affected planes, expected fail-closed behaviour, and audit semantics for structured comparison. The *acceptance protocol* is realized in an open-source *artefact*: hypotheses H1–H6 are supported by tagged automated regression (nine cases in the paper replication subset) and by governance profiles (none, weak, full); the mapping from hypotheses to criteria appears in Appendix A, Table 4. Supplemental stressors are exercised through additional release-gate scripts. Section 10.2 summarises scripted ablations reinforcing interpretability claims. Claims are *protocol- and harness-level*; a single successful replication run does not certify statistically representative exposure in production deployments.
5. **Governed extension points.** Connectors, `MemoryBackend` implementations (including optional Mem0-class adapters), `OrchestrationEngine` backends (e.g., native directed acyclic graph execution versus graph runtimes), evolution plugins, and scheduler bridges are composable replacements behind stable interfaces—the objective is to inter-

change *implementations*, not to host a parallel permission regime exempt from manifests and policy. Contractual surfaces (`run_manifest`, `PolicyEngine`, tenancy isolation, and joinable audit and proposal chains keyed by `run_id`, `tool_use_id`, and policy references) must be preserved under substitution: outward writes and sensitive promotions must not become silently unconstrained or forensically opaque owing to adapter choice; trajectory-shaped learning inputs surface only as `ProposalQueue` entries and analogous governance queues, never as silent overwrites of production policy bundles or manifests. The reference prototype centralizes structured selections for orchestration, memory, and evolution in a single version-controlled YAML artefact; environment variables chiefly supply configuration paths and secret references, reducing inconsistent dual configuration between shell environments and policy artefacts.

6. **Ingress protocol and grounding hooks.** Mandatory ingress fields elevate HTTP control-plane payloads beyond bare tool identifiers and arguments: `message_disposition` (`collect`, `followup`, `steer`) specifies how an inbound turn is to be classified for routing, policy, and audit consistently across channels in the artefact evaluated here; `mcp_execution_tier` (`direct-tool`, `code-exec-mcp`) reflects host/tool exposure distinctions Anthropic publishes in MCP documentation [1] and fixes the permitted execution modality at our boundary, discouraging implicit escalation after admission. Together with `run_id`, optional `run_manifest`, and trace identifiers, these fields mitigate inconsistent enforcement between interactive and programmatic clients. *Grounding hooks* permit research- or analysis-oriented responses to carry structured `references` coextensive with `SourceRef` (`source_kind`, `locator`, optional `tenant_scope`, `excerpt_policy`), anchoring claims to permission-scoped evidence rather than to unattributed prose. We specify these elements as versioned ingress and response contracts and explicitly do *not* assert that a single software revision fully instantiates every staged data-loss-prevention obligation across all integration paths.

2 Related Work

We group prior work by concern—tool-mediated action, gateways and assistants, hardened open runtimes, and governance-facing evaluation—then position Agentium relative to recurrent themes without claiming exhaustive feature parity.

Citation and evidence scope. Claims about **this prototype** (field names, pipelines, test counts) trace to the artefact at the repository URL

on the title page and revision cited in harness outputs. **Third-party** OSS projects are described only as far as the `references.bib` entries (URLs or papers) support; we do not guess behaviour absent from those sources. Closed products in Table 1 use the same rule: cells paraphrase only the cited vendor posts [5, 18].

2.1 Foundations: tool use and reflective agents

ReAct [31] interleaves reasoning and tool calls at the language-model boundary; complementary learning formulations (Toolformer [24]; Reflexion [25]) predate today’s enterprise-grade governance regimes. A survey consolidates tool-augmented model families [15]. Agentium assumes such capabilities exist and concentrates on imposing versioned manifests, gates, and auditable linkage—not on improving latent tool-use accuracy per se.

2.2 Gateways, protocols, and multi-channel assistants

OpenClaw emphasises a long-lived gateway, typed protocols, idempotency, and diverse messaging surfaces in its public architecture documentation [20, 21]. The Model Context Protocol is documented and evolved by Anthropic as a publicly available specification for host–tool context exchange [1]; complementary software-engineering work analyses verifiable tool mediation for hosted agents [7]. Agentium shares control-plane hygiene but foregrounds organisational policy versioning, tenancy, immutable manifests, and joinable audit alongside routing—so that gateway behaviour and ingress contracts do not drift from enforcement.

2.3 Super-agent harnesses, product assistants, and industrial practice

DeerFlow and related stacks centre long-running super-agent harnesses, sub-agents, and sandboxes [6]. Proprietary assistants such as ChatGPT GPTs and Claude Code [5, 18] are cited only via *voluntarily disclosed* marketing or introductory engineering posts; absent peer-reviewed artefacts or independently reproducible gateways, Table 1 must *not* be read as claims about undocumented internals (we neither assert nor deny specific controls hidden behind closed implementations). Industrial practice from foundation-model providers stresses context budgets, harness structure, and evaluation discipline [2, 4]; we transpose those concerns into explicit runtime and ingress contracts.

2.4 Specialised open stacks and personalised memory backends

EdgeClaw, IronClaw, and Hermes-style agents layer memory, routing, or safety-first runtimes [16, 17, 19]. Agentium’s emphasis is cross-cutting governance and audit rather than a single narrow technique (e.g., WASM-only isolation). Personalized memory services such as Mem0-class stacks [14] illustrate demand for pluggable recall layers. Benchmarks such as SWE-bench evaluate LLM-produced patches on executable repositories [8]; we cite them only as motivation that tool-mediated changes to real codebases merit auditable governance boundaries—not as evidence of our performance on those benchmarks. We treat backends as interchangeable implementations whose durable writes must align with manifests and policy (Section 5).

2.5 Surveys, taxonomies, and governance-aligned enforcement

Surveys of LLM-based autonomous agents [29] and taxonomies contrasting *AI agents* with *agentic AI* [23] situate the field; our work stresses operator-grade control planes and reproducible gates in live deployments. Frameworks for ethical assurance and governance of agentic systems [9], runtime enforcement for safe and reliable LLM agents [28], and verifiably safe tool invocation [7] align with manifest and policy boundaries; classic end-to-end arguments for safeguarding shared systems [22] inform our segregation of consequential planes. Design-time taxonomies and staged assurance arguments become operationally meaningful when runtime queues (e.g., asynchronous `ProposalQueue` state transitions) emit joinable audit events associated with tenants and runs.

2.6 Evaluation, scaling, and orchestration perspectives

Evidence-synthesis perspectives for evaluating and governing agentic AI [11], scaling properties of agent systems [10], utility-guided agent orchestration for tool use [13], and protocol-level multi-agent stacks such as AgentOrchestra [32] motivate budgeted coordination and observability. Complementary analyses of multi-agent versus single-agent performance under controlled conditions [26, 27] further support explicit manifests and scheduling rather than unconstrained agent teams.

2.7 Qualitative positioning

Table 1 summarises, across four substantive axes, where the Agentium prototype emphasises contractual mechanisms in qualitative contrast to themes visible in publicly documented OSS stacks [6, 20, 21]. Closed products such as GPTs [18] and Claude Code [5] are deliberately *excluded* from the

cells: their public introductions do not document behaviour along these axes, and we neither assert nor deny undocumented controls; qualitative remarks on those products, when relevant, stay in prose and cite only the vendor posts.

Table 1: Qualitative positioning (illustrative; not a statistical benchmark). *Primary* denotes the evaluated Agentium artefact only; other columns paraphrase *open* docs for the OSS stacks and are neither reverse-engineering claims nor feature certifications.

Axis	OpenClaw	DeerFlow	Agentium
Run-scoped tool & policy contract	partial (per cited docs)	partial (per cited readme)	primary
Audit replay by run and tool use	partial	varies by deployment	primary
Declarative configuration of plugins	varies by deployment	varies by deployment	primary
Learning → governed proposals only	not documented	not documented	primary

Closest-system distinction. Agentium is not offered as a drop-in replacement for a gateway or a research harness: the contribution is *co-designed* control surfaces—immutable manifests, joint policy and human approval with joinable audit, a governed background plane, and versioned ingress semantics (Section 5 and ingress discussion in Section 4)—together with automatable acceptance criteria (Section 9, Appendix A). Third-party stacks may implement subsets; the manuscript’s claim is *joint* obligations and falsifiable harness checks, not novelty of any single mechanism in isolation.

3 Threat model and security goals

Scope and assurance stance. The threat model fixes *which* classes of misuse we analyse, *which* assets are in play, and *which* attacker capabilities remain out of scope. The manuscript claims *architecture-level containment* and *joinable observability*: when gateways, manifests, registry, policy, and audit components behave as specified, externally consequential acts are mediated through declared surfaces and attributable records suitable for replay and governance review—not cryptographic proof against arbitrary physical or cryptographic adversaries.

Explicitly excluded from this manuscript’s assurance claims are: physical attacks against operators or facilities; covert side-channel leakage between tenants through channels other than documented interfaces; unrestricted

compromise of cryptographic keys underpinning transports; and unconditional semantic correctness or truthfulness of free-form LLM prose (except where egress or policy intersects analysed text).

Assets and stakeholders. Tenants own budgets, memory, and artefacts; each `run_id` / session is an instance subject to quotas; operators require approvals, rollbacks, and forensics tied to manifests.

Adversaries and misuse. We consider prompt-induced misuse and social-engineering style escalation paths, buggy configuration widening capabilities relative to manifests, abusive background workloads, and cross-tenant confusion on shared infra. Adversaries need not corrupt data centres outright: inducing a model-assisted call through an authorised API with tool arguments shaped to exfiltrate data, or exploiting mis-parameterised tenancy fields, suffice as *threat compositions* targeted by the gates below.

We *do not* assume a nation-state cryptanalytic attacker; assurance is conditional on faithful component behaviour.

Security goals. Default deny on undeclared tools and cross-tenant reads; observable decisions tying tool calls to policy versions and traces; mandatory human oversight for designated high-risk classes; graceful degradation under fault with bounded recovery targets.

Illustrative misuse compositions. The following vignettes are *illustrative*—they refine intuition for Appendix A’s exercised criteria and do not introduce new mechanistic assumptions.

- **Manifest mismatch.** A run declares only read-oriented tools; adversarial prompting or erroneous tool planning nonetheless requests undeclared side-effecting tools. The acceptable outcome under this model is refusal with audit, not silently expanded capability.
- **Tenant boundary.** Queries or artefacts are addressed with an incorrect tenant scope (buggy client, confused route, or flawed join key). Acceptable behaviour is explicit failure or empty legal results tied to tenancy metadata—not silent cross-tenant reads.
- **Background amplification.** Event storms or duplicated triggers inflate outbound notifications or outbound tool use without proportional oversight. Controls should bound outward effects via throttles or kill-switch semantics aligned with Sections 9 and 10.

4 System design

Planes. Governance hosts `PolicyEngine`, `ApprovalGate`, lineage; coordination hosts DAG/workflows, artefacts, budgets, emergence guardrails; background hosts triggers/daemons gated by policy; control hosts scheduling and metrics; execution hosts runtimes and tool registry; interfaces stay thin [2].

Control vs. execution.

Mirroring kernel/user separation, budgets and manifests are adjudicated centrally; harness loops execute reasoning but outward effects traverse audited hooks only.

Multi-agent coordination.

Task graphs expose idempotency keys, leases, and timeouts so shared resources do not stall indefinitely.

Ingress contracts and data-loss prevention staging. **Message disposition** is encoded as an enumerated ingress field (`collect`, `followup`, `steer`) stating how an inbound turn is to be interpreted for routing and audit. **MCP execution tiers** as used here are informed by Anthropic’s published MCP documentation on host/tool exposure [1], distinguishing `direct-tool` invocation from `code-exec-mcp` paths with differing trust and exposure assumptions imposed by our ingress contract. HTTP ingress maps these fields into `RequestContext` and emits `turn_ingress` audit events so that heterogeneous clients cannot diverge silently in semantics. Data-loss prevention (DLP) is modeled as a multi-stage pipeline; in the evaluated prototype, tool-output inspection after handler return is labeled `dlp_stage=tool_output_post` on `dlp_hits_detected`/`dlp_blocked` audit rows, whereas full coverage of ingress, pre-context, pre-tool, post-language-model, and egress stages remains an incremental objective. Research-oriented responses may include structured **references** aligned with `SourceRef` without displacing natural-language explanation.

Figure 1 summarises the control-centric decomposition used throughout the design.

5 Governed Extension Points: Persistence, Memory, Background, and Learning

The architecture permits substitution at the infrastructure boundary: deployments may exchange SQL-compatible stores, attach third-party memory systems, replace daemon or scheduler substrates, and integrate offline adaptation modules. The scientific claim is not a particular embedding algorithm or scheduling heuristic; rather, **no substitutable module may weaken contractual surfaces**—manifests, policy versions, tenancy partitioning, and

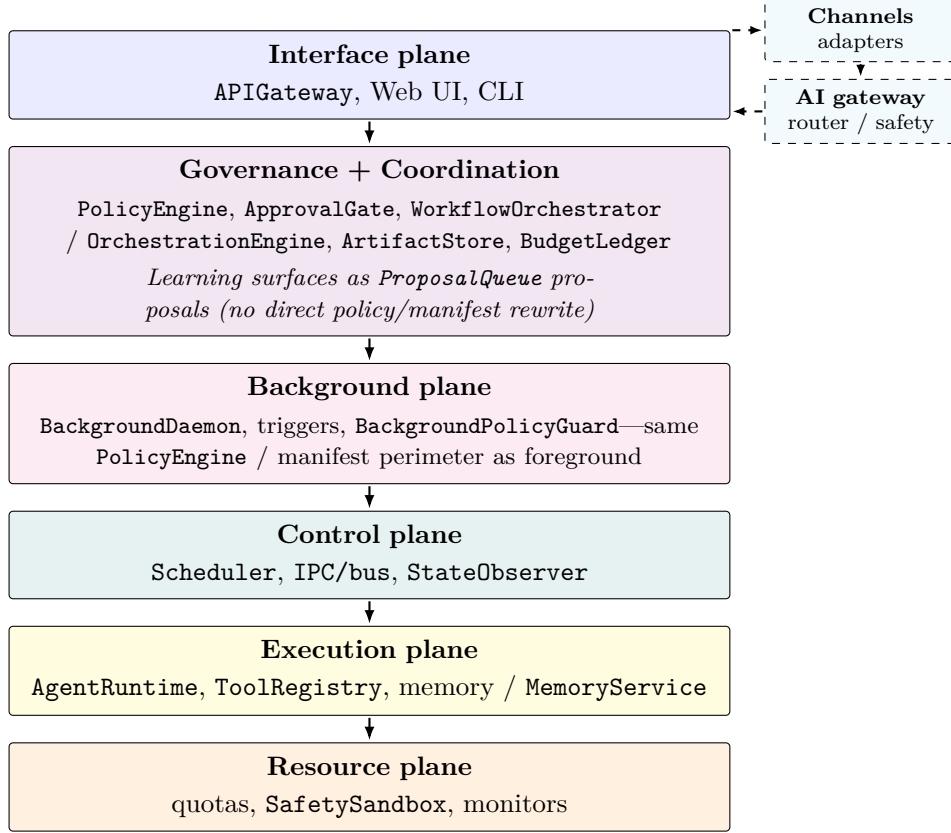


Figure 1: Conceptual layering of Agentium planes (interface through resource). Dashed boxes indicate channel adapters and an AI gateway that feed ingress; the **Background plane** is shown explicitly under governance. Component labels are schematic.

audit joins must remain invariant across backend choices. In the reference prototype, structured selections for orchestration engines, memory backends (including optional external client integrations), and evolution plugins are loaded from a *single version-controlled configuration document*; environment variables chiefly indicate configuration paths and secret references rather than duplicating per-plugin toggles. Workflow execution can be factored behind an **OrchestrationEngine** abstraction (native runner versus optional adapters such as LangGraph [12]) while preserving the same **WorkflowSpec**, artefact contracts, and human-in-the-loop resume semantics enforced by the core orchestrator. Trajectory-shaped inputs to adaptation plugins are emitted automatically after **DeepResearchPipeline** completes (bounded summaries: per-node status and output keys, not full model outputs) and may additionally be submitted through an HTTP surface when explicitly

enabled in configuration and gated by capability checks; in all cases learning continues to surface only `ProposalQueue` entries, never direct rewrites of production policy bundles or manifests.

Composable backends under contract. Connectors for databases, caches, queues, or object stores inherit the same tenancy and lineage obligations as core services: schemas may differ, but attribution fields (`run_id`, tenant scope, policy snapshot identifiers) remain joinable for forensics. This avoids the failure mode where a “drop-in” store bypasses organisational controls because integrations were coded as ad hoc data sinks.

Governed memory backends. Long-term memory layers (whether bespoke indices or externally maintained retrieval services such as Mem0-class backends [14]) are treated as subordinate writers: retrieval can be rich and low-latency, but durable updates that change tenant-visible knowledge must pass through classified write paths governed by policies and manifests—not unchecked model-mediated overwrites of canonical facts. Promotion of segments into retained corpora—including documentation-style corpora maintained for agents in the spirit of curated “wiki” workflows—can be configured to require review or dual control when impact or blast radius crosses configured thresholds. The evaluation protocol targets gate semantics and tenancy (Section 10); ranking quality of a particular embedding backend remains out of scope unless tied to audits. The open-source artefact provides a `MemoryBackend` abstraction with two built-in backends (in-memory, SQLite) and an experimental `Mem0Backend` adapter exemplifying integration of an external Mem0-class client without relaxing `MemoryService`-level tenancy isolation; backend selection uses the same plugin configuration surface described at the opening of this section.

Daemon and scheduler substitution. Background automation may be driven by purpose-built workers or bridges to external agent runtimes that expose scheduling APIs; the contract is unchanged: triggers stay on the governed background plane, inherit circuit-breaking and rate limits, and cannot register tools or egress outside the active manifest. Commercial or research schedulers are not normative here—only their observability and failure semantics under policy matter.

Adaptation and self-improvement. Learning loops (preference updates, reward sketches, candidate policy deltas, or prompt-template proposals) are framed as **proposals** entering governance queues rather than silent self-modification of production parameters. High-stakes changes require explicit approval or staged rollout with rollback; audit records remain compatible with incident review. The reference prototype instantiates this pattern in

a `ProposalQueue` component whose kind enumeration covers memory promotion, policy deltas, prompt templates, and preference updates; each state transition emits a `proposal_state_changed` audit record. Long-horizon open-research agents that stress continuous improvement [17] illustrate why ungated adaptation is risky at enterprise scale; we emphasise operator control over *when* learning alters externally visible behaviour.

Tiered autonomy versus human oversight. Requiring a human keystroke on every retrieval or internal inference step would be impractical; conversely, allowing unconstrained autonomy for externally consequential actions defeats the control-plane premise. Agentium distinguishes **risk tiers**: actions already authorized by the active manifest and policy may execute automatically; classes designated as irreversible writes, sensitive egress, cross-tenant paths, broad knowledge-base promotion, or destructive background operations defer to synchronous human gates or asynchronous review workflows. Human intent appears both declaratively—authored manifests and policies versioned under change control—and interactively via approvals [9], so autonomy is delegated rather than absolute.

6 Immutable run contract: operational semantics

This section distils the *immutable run contract* obligations that Section 4 presupposes. The goal is a compact, falsifiable reading of what “run-bound” means for evidence: which records must exist, which transitions are legal, and where fail-closed defaults apply. Section 7 then illustrates the resulting control path (Figure 2). We state semantics at the level of the reference prototype; formal verification of a narrowed ingress-registry protocol is left to future work.

Contractual objects. A *run* is identified by `run_id` scoped to a tenant and correlated with `request_id`, `trace_id`, and optional `run_manifest` digests propagated into `RequestContext`. A *manifest snapshot*, when supplied, declares an allow-listed tool universe (`declared_tools`) and optional immutable references (`policy_bundle_ref`, build metadata). Policy adjudication binds `PolicyEngine` releases to tool names and roles. Human gates bind decisions at `tool_use_id` granularity with argument hashes.¹

Ingress transition. **Ingress** accepts or rejects admission of an inbound turn carrying optional manifest payloads. Fail-closed codes include absent manifests when mandated, mismatched profiles, mismatched cryptographic

¹Ingress records `message_disposition` and `mcp_execution_tier` as orthogonal routing facets; altering them mid-run without revisiting manifests is forbidden by ingress validation in the artefact evaluated here.

digests, and divergent policy bundle refs relative to deployment defaults (surfaced as auditable mismatch warnings tied to manifests) *when those checks are enabled in configuration*. Accepted runs inject allow-list constraints into downstream execution contexts.

Tool dispatch. Manifest alignment applies before policy rules: if `manifest_declared_tools` is present, any tool $t \notin \text{declared_tools}$ deterministically raises a denial with `run_manifest_tool_denied` lineage. This step is independent of policy “optimism”; it enforces the manifest contract even when policy could otherwise allow the tool class.

Policy and human gates. Policy adjudication maps (tenant, role, tool, args) to `allow`, `require_approval`, or `deny`, emitting versioned audit rows. **Approval resume** requires that an `approval_id` matches pending or approved state, argument hashes, and tool identity; rejected or expired approvals cannot transition to successful execution. Natural-language operator rationale is not a substitute for these mechanical checks.

Background equivalence. Background-plane actions inherit the same policy and manifest contexts as interactive turns: there is no alternate “shadow” permission model for scheduled automation in the evaluated configuration.

Documented invariants (sketches).

- **I1 (manifest monotonicity).** Once recorded for a run, manifest digests and declared tool sets used for enforcement are not silently widened without a new admitted manifest and corresponding audit.
- **I2 (fail-closed tool paths).** Undeclared tools never reach handlers when manifest allow-lists are active *in the evaluated `ToolRegistry` path*.
- **I3 (approvals bind structure).** Successful high-risk execution requires approval artifacts whose identifiers and argument hashes match the resumed attempt.

These sketches align with Figure 1 and Section 9, and ground the single-sentence contribution claim that contracts, gates, and audits are co-designed rather than bolted on post hoc. Figure 2 in Section 7 sketches the same chain at the level of concrete components (`ToolRegistry`, `PolicyEngine`, `Audit`, `Execute`).

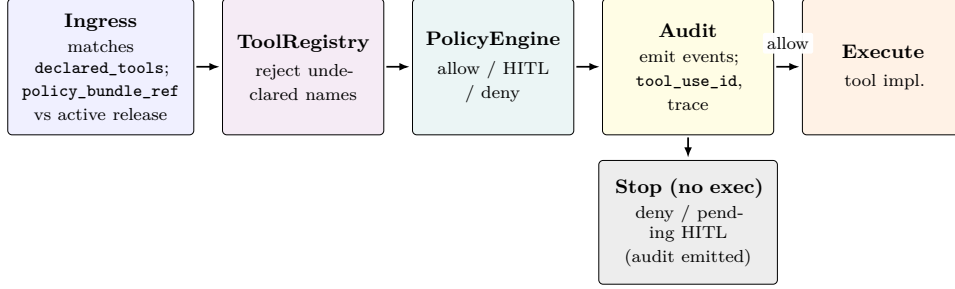


Figure 2: Simplified tool-use gate: ingress aligns declared tools and `policy_bundle_ref` with the active policy release; `ToolRegistry` and `PolicyEngine` gate execution. **Audit** emits events for all outcomes; only the *allow* path reaches **Execute**. Outbound data loss prevention (hypothesis H4) follows a distinct egress path and is not depicted in this sequence.

7 Container-style operational mechanisms

We adapt *container-style* semantics into engineered constraints—the analogy is structural, not a dependence on Docker itself: immutable snapshots (`run_manifest`); template vs. instance separation; tenant-scoped quotas reminiscent of cgroup limits; artefacts as volumes; restart and kill-switch behaviours; signed policy artefacts. Ingress validates manifests and injects allowed tool names before tool dispatch so undeclared names fail closed. A manifest’s `policy_bundle_ref` is additionally compared at ingress against the active policy release; divergence emits a warning-level `policy_bundle_ref_mismatch` audit event so replay traces can always be bound to the version that actually executed the run.

Figure 2 abstracts H1-style blocking (`run_manifest_tool_denied` on undeclared tools) layered under policy semantics; the invariants that it realises at the level of records are stated in Section 6 (I1–I3).

8 Control-Theoretic Safety Loop

We adopt a control-theoretic stance favouring end-to-end back-pressure (channels through the model gateway), measurable admission control rather than unbounded queues, human oversight on slow paths, conservative adoption of heavyweight formal methods, and explicit observability of disturbance classes (overload, misuse, drift). Theoretical machinery is introduced only where threshold-based staging permits managed complexity.

Loop stages mapped to observables. Figure 3 is schematic; the five stages map to concrete operational objects rather than to model-inferred

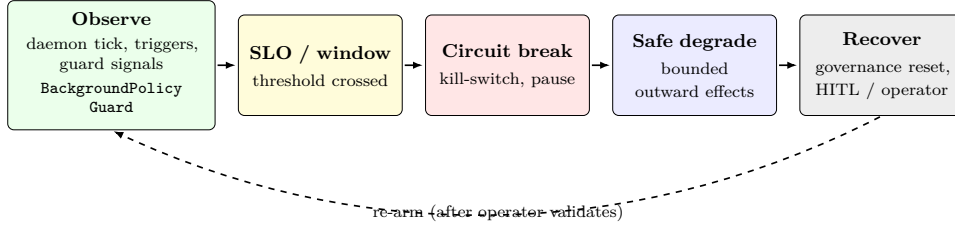


Figure 3: Schematic background-plane safety loop: observation closes on noisy or faulty triggers; breaching thresholds activates circuit-breaking and kill-switch semantics before returning to nominal operation under governance-controlled recovery (hypothesis H3-style bounded degradation).

policies.

- **Observe** — daemon ticks, trigger invocations, and **BackgroundPolicy Guard** signals are accumulated over deployment-configured windows; no model text is used as a control input.
- **SLO / window** — deployment parameters (rate limits, error ratios, MTTR budget) define the threshold surface; crossings are recorded on the background plane’s audit trail rather than inferred.
- **Circuit break** — operational controls (throttles, pauses, kill-switch) are engaged; no new tool admissions or outbound side effects occur on the guarded path until the loop resets.
- **Safe degrade** — outward effects are bounded by configuration (disabled channels, queued proposals rather than direct writes); tenant and manifest contracts remain in force.
- **Recover** — re-arm is gated by governance (operator or HITL validation), not by automatic retry; this is the H3 drill anchor recorded through the same audit surface.

Thresholds and timers are deployment parameters tied to audit logs; Section 9 summarises how the same stages are exercised under the H3 drill hooks.

9 Hard Acceptance Criteria

We summarise six automated gates exercised as *regression and release-gate harnesses* in the prototype: undeclared tool block with audit trails; outbound DLP on representative synthetic sensitive payloads; unconditional

cross-tenant denial with explicit failures in tagged tests; noisy background-plane scenarios tripping safeguards and recovery paths; no successful high-risk approval bypass in the tagged approval-integrity tests; convergence to intentional safe-degraded states within a configured MTTR envelope in drill hooks.

Table 2: Narrative summary of the six gates and their primary anchoring mechanics in the prototype. Appendix A, Table 4 records the hypothesis identifiers (H1–H6) and replication-test roles.

Gate (conceptual)	Primary anchoring mechanics
Undeclared tools refuse with audit	Ingress-aligned manifest injection; registry allow-list; denial events keyed to manifests
Outbound sensitive-payload interception	Instrumented data-loss-prevention pathway and audit-labelled stages where implemented
Cross-tenant access fails closed	Tenancy propagation in retrieval and artefacts; explicit error semantics
Background mis-triggers guarded; recovery exercised	Background policy guards; triggers under rate limits; circuit-breaking and drills
Approval bypass prevented for designated risk	<code>PolicyEngine+ApprovalGate</code> chain at <code>tool_use_id</code> granularity
Safe-degraded operation within bounded MTTR	Operational drill hooks; recovery thresholds tied to observable state

10 Evaluation Methodology and Automated Verification

This section states harness-level methodology. Procedural detail and the hypothesis–verification map are given in Appendix A, Table 4.

We evaluate the reference prototype using automated regression executed through a bundled *replication driver*. The driver records a non-secret environment fingerprint (plugin *selection* identifiers, interpreter metadata, version-control revision), executes automated tests including a *paper replication subset* of nine cases that jointly witness hypotheses H1–H6 (hypothesis H2 is exercised by three tests), runs three governance profiles (none, weak, full—homologous to `allow`, `require_approval`, and `deny` in the evaluated `PolicyEngine`), and invokes supplemental release-gate scripts for staged workloads (e.g., frequency controls, emergence guardrails, credential-phishing patterns). It emits a machine-readable summary with exit statuses and version-control metadata. Table 4 relates each hypothesis to its criterion and verification role; interpretive limits follow.

We characterize this evaluation as *protocol and harness verification* with preliminary metrics (e.g., gate pass/fail and count-based aggregates from scripted hooks), not as evidence of statistically representative behaviour in production deployments. The harness foregrounds tenancy, manifest enforcement, approvals, data-loss-prevention instrumentation, and background safeguards—not an ablation over memory-embedding backends, which Section 5 treats as interchangeable behind a common ingress.

Scope of verification claims. Appendix A, Table 4 maps **hypotheses H1–H6** to six *automatable acceptance criteria*: manifest-aligned tool denial, bypass-free approvals, guarded background behaviour under stressors, outbound data-loss-prevention interception, explicit cross-tenant denial, and recovery-oriented drills. Those identifiers are separate from **ingress and response contracts** enumerated in Section 4 (`message_disposition`, `mcp_execution_tier`, structured `references/SourceRef`), which we state as protocol obligations and partially exercise via the artefact without claiming turnkey, corpus-scale statistical grounding for unrestricted research workloads within the replication subset.

10.1 Threats to validity

We structure limitations following common software-engineering experiment reporting [30].

Construct validity. Acceptance hypotheses H1–H6 operationalise enterprise concerns through scripted tests and deterministic stimuli: they measure whether the artefact denies or audits specific actions, *not* whether workloads statistically resemble production intrusion campaigns or nuanced insider threats. Controlled ablation similarly toggles only evaluation-guarded weakening hooks (manifest enforcement and policy coercion) inside the toolkit; behaviours outside `ToolRegistry` (e.g., background daemons that never consult the same code path) are intentionally out of scope for those toggles.

Internal validity. Results depend on faithful configuration of the replication driver, tagged tests, and evaluation-only environment variables that must remain disabled in operational deployments. Drift between documentation and code would threaten repeatability; we mitigate this by machine-readable summaries, version-control fingerprints, and explicit mapping files checked into the repository (Appendix B).

External validity. H5 explicitly exercises cross-tenant denial; the remaining hypotheses H1–H4 and H6 run under default single-tenant configurations in the packaged harness. DLP uses synthetic secret-shaped payloads; the

evaluated harness does not cover diverse LLM providers or unconstrained natural-language attacks. Microbenchmarks measure classifier wiring overhead on benign echo workloads, not representative tail latency under adversarial traffic. Claims must not be extrapolated to fleet-scale SLO proofs without independent measurement.

10.2 Controlled ablation and microbenchmark

Complementing the regression bundle, we script a three-way *synthetic harness* atop the tagged paper subset (nine tests). Configurations isolate loss of manifest allow-list discipline versus loss of restrictive policy coercion while keeping ancillary controls (cross-tenant memory checks, deterministic drills, DLP path for H4, background guard) untouched—see Appendix B for procedural detail and artefact pointers. Expected failures concentrate on hypotheses H1 and H2 for the degraded configurations, furnishing interpretable contrasts without invoking proprietary models.²

A companion microbenchmark (`scripts/ablation_microbench.py`) times repeated executions of a registered low-risk echo tool through `Tool Registry`: one stack omits outbound classifiers (`dlp_classifier=None`); the identical stack attaches `DLPClassifier` at the audited hook so benign payloads still exercise classifier wiring. It records `p50_ms`, `p95_ms`, and `mean_ms` (via `time.perf_counter_ns`) under JSON keys `minimal_stack` and `with_dlp_stack`, bundled as `microbench.json` and embedded under `microbench` in `ablation_summary.json` (Appendix B). Printer-fixed latency tables are omitted because numbers are intentionally environment-specific; they are nonetheless reproducible from the artefacts.

Table 3: Synthetic ablation factors (nine tagged paper tests plus microbenchmark). “Expected failing hypotheses” denote tests engineered to observe regressions relative to Full.

Configuration	Operational meaning	Expected failing hypotheses (subset)
Full / control	Artefact-default enforcement	none (pass all tagged tests)
No-manifest (eval. only)	Allow-list discipline disabled in <code>Tool Registry</code>	H1 manifests tests
Permissive policy (eval. only)	Policy outcomes coerced to <code>allow</code> after rule evaluation	H2 approval-integrity trio

²Closed assistants are irrelevant to harness execution; qualitative positioning cites only public introductions and avoids undocumented feature contrasts.

Data and software availability. The open-source artefact underpinning Sections 10 and A is publicly hosted at <https://github.com/GuiminChen/agentium>. For reproducibility, authors recommend citing—or pinning—a concrete version-control revision or release tag alongside any reported harness summaries; replication drivers emit fingerprints that include interpreter metadata and selections, omitting credentials.

11 Discussion

Binding security properties to named components increases implementation obligation in exchange for improved defensibility. Workflow-centric deployments in high-assurance settings remain compatible with elevated autonomy when supporting evidence exists. Observable metering is prerequisite to closed-loop cost or latency control. Governed extension points (Section 5) permit adoption of specialised memory or scheduler substrates without branching the policy model, at the cost of preserving connector invariants—a deliberate trade-off between flexibility and integration discipline.

Beyond a gateway plugin. Public OSS stacks such as OpenClaw foreground a long-lived *gateway* with typed protocols and multiple messaging surfaces [20, 21]; DeerFlow-class systems foreground long-horizon research *super-agent harnesses*, sub-agents, and sandboxes [6]. Those emphases address connectivity, scale, and iterative task structure. Agentium treats **the same ingress edges**—channel adapters and an AI gateway feeding a common HTTP control-plane path (Figure 1)—as *necessary but insufficient* for organisational operation: the additional binding is **versioned manifests, policy bundles and human gates** with joinable audit, plus a **governed background** plane that cannot relax the same perimeter silently. The artefact therefore remains composable with gateway-style plugins and channel diversity; the manuscript’s claim is not to replace routing, but to keep routing, tool registration, egress, and automation on **one contractual and auditable spine** so that “pluginisation” does not reintroduce divergent enforcement between clients or between foreground and triggers.

Operational trade-offs. Mandatory human approvals and staged data-loss prevention increase latency on paths that trigger them and may inconvenience benign payloads at configurable false-positive rates; kill-switch semantics improve blast-radius containment at the price of degraded automation until operators explicitly recover. Deployments should profile human queue depth, blocker rates, and DLP escalation volume alongside purely technical metrics.

12 Limitations and Future Work

Prototype coverage is incomplete relative to the full architectural design; empirical claims remain deliberately circumscribed, with explicit threats enumerated in Section 10.1. The threat model excludes physical and cryptographic side channels. We bound tool and egress policy, not general semantic correctness of model-generated text except where data-loss prevention applies. Persisted-memory backends and adaptation pipelines discussed in Section 5 are treated at the level of contractual fidelity; comparative benchmarking of retrieval quality against Mem0-class or retrieval-only systems awaits future controlled studies. Structured **references** grounded in permission-scoped **SourceRef** fields constitute a protocol and tooling target for research-oriented responses; estimating coverage fractions, sampling plans, or human verification rates on curated corpora is left for future controlled studies rather than asserted here beyond artefact scaffolding. The evolution pathway bundled with the artefact routes bounded trajectories to **ProposalQueue** proposals only and does not instantiate a full multi-step “Evolver”-style engine as described in some third-party stacks. Future work includes larger-scale multi-tenant studies, stronger external baselines, optional formal verification of narrow protocols, and empirical comparison of gated memory-promotion workflows under identical manifests.

13 Conclusion

Enterprise LLM-agent operation fails when tool use, egress, and background automation drift from declared policy and from each other. Agentium binds them to one contractual spine—immutable run manifests, joint policy and human gates, auditable background automation, and versioned ingress semantics—formalised as I1–I3 (Section 6) and exercised as H1–H6 (Appendix A). Evidence is *harness-level and artefact-bound*: the architecture supports auditable, rollback-capable operation without claiming benchmark superiority or statistically representative production behaviour.

References

- [1] Anthropic. Introducing the model context protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Vendor-maintained specification and docs for model–tool context; not an IETF/W3C consensus standard.
- [2] Anthropic. Building effective agents. <https://www.anthropic.com/engineering/building-effective-agents/>, 2025. Industry engineering article.

- [3] Anthropic. Effective context engineering for AI agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents/>, 2025.
- [4] Anthropic. Effective harnesses for long-running agents. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents/>, 2025.
- [5] Anthropic. Introducing Claude Code. <https://www.anthropic.com/news/claude-code>, 2025. Product brief; qualitative positioning relative to OSS gateways.
- [6] ByteDance. Deerflow. <https://github.com/bytedance/deer-flow>, 2025. Open-source super-agent and research workflow stack.
- [7] Aarya Doshi, Yining Hong, Congying Xu, Eunsuk Kang, Alexandros Kapravelos, and Christian Kästner. Towards verifiably safe tool use for LLM agents. arXiv preprint arXiv:2601.08012, 2026. URL <https://arxiv.org/abs/2601.08012>. Accepted to ICSE NIER 2026 per arXiv comment; primary class cs.SE.
- [8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- [9] Rafflesia Khan, Declan Joyce, and Mansura Habiba. AGENTS SAFE: A unified framework for ethical assurance and governance in agentic AI. arXiv preprint arXiv:2512.03180, 2025. URL <https://arxiv.org/abs/2512.03180>. arXiv identifier 2512.* denotes a Dec 2025 submission month; cite revision used.
- [10] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a science of scaling agent systems. arXiv preprint arXiv:2512.08296, 2025. URL <https://arxiv.org/abs/2512.08296>.
- [11] Christopher Koch and Joshua Andreas Wellbrock. Beyond task success: An evidence-synthesis framework for evaluating, governing, and orchestrating agentic AI. arXiv preprint arXiv:2604.19818, 2026. URL <https://arxiv.org/abs/2604.19818>. Primary class cs.SE.
- [12] LangChain. Langgraph. <https://github.com/langchain-ai/langgraph>, 2025. Graph-based orchestration library for agent workflows.

- [13] Boyan Liu, Gongming Zhao, and Hongli Xu. Utility-guided agent orchestration for efficient LLM tool use. arXiv preprint arXiv:2603.19896, 2026. URL <https://arxiv.org/abs/2603.19896>.
- [14] Mem0. Mem0: Memory layer for personalized AI. <https://github.com/mem0ai/mem0>, 2025. Open-source episodic-memory stack cited as illustrative third-party backend.
- [15] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. Augmented language models: a survey. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=jh7wH2AzKK>.
- [16] NearAI. Ironclaw. <https://github.com/nearai/ironclaw>, 2025. Safety-oriented local agent runtime.
- [17] Nous Research. Hermes agent. <https://github.com/NousResearch/Hermes-Agent>, 2025.
- [18] OpenAI. Introducing GPTs. <https://openai.com/index/introducing-gpts/>, 2023.
- [19] OpenBMB. Edgeclaw. <https://github.com/OpenBMB/EdgeClaw>, 2025.
- [20] OpenClaw. Gateway architecture. <https://docs.openclaw.ai/concepts/architecture>, 2025.
- [21] OpenClaw contributors. Openclaw. <https://github.com/openclaw/openclaw>, 2025. Open-source agent gateway and messaging surface.
- [22] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [23] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges. *Information Fusion*, 120:103599, 2025. doi: 10.1016/j.inffus.2025.103599. URL <https://arxiv.org/abs/2505.10468>. Also arXiv:2505.10468.
- [24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL <https://arxiv.org/abs/2302.04761>.

- [25] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [26] Junwei Su and Chuan Wu. When do multi-agent systems outperform? analysing the learning efficiency of agentic systems. arXiv preprint arXiv:2602.08272, 2026. URL <https://arxiv.org/abs/2602.08272>. Primary class cs.LG.
- [27] Dat Tran and Douwe Kiela. Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. arXiv preprint arXiv:2604.02460, 2026. URL <https://arxiv.org/abs/2604.02460>.
- [28] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. arXiv preprint arXiv:2503.18666, 2026. URL <https://arxiv.org/abs/2503.18666>. arXiv identifier 2503.* is first posted Mar 2025; ICSE 2026 acceptance per arXiv abstract text—confirm proceedings meta at camera-ready.
- [29] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1. URL <https://arxiv.org/abs/2308.11432>. Journal uses electronic article number; corroborating preprint history at arXiv:2308.11432.
- [30] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012. doi: 10.1007/978-3-642-29044-2.
- [31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [32] Wentao Zhang, Liang Zeng, Yuzhen Xiao, Yongcong Li, Ce Cui, Yilei Zhao, Rui Hu, Yang Liu, Yahui Zhou, and Bo An. AgentOrchestra: Orchestrating multi-agent intelligence with the tool-environment-agent (TEA) protocol. arXiv preprint arXiv:2506.12508, 2025. URL <https://arxiv.org/abs/2506.12508>. Stable citation should pin the arXiv version accessed.

Appendix

A Replication bundle and hypothesis–verification map

Bundled workflow. The open-source artefact includes an executable *replication driver* that performs, in order:

1. records a non-secret *environment fingerprint*—version-control revision, interpreter metadata, and active plugin *selection* identifiers—excluding credentials;
2. executes automated regression tests, including a *paper replication subset* of nine cases that jointly satisfy acceptance hypotheses H1–H6 (hypothesis H2 is covered by three tests);
3. evaluates three governance profiles (`none`, `weak`, `full`), homologous to `allow`, `require_approval`, and `deny` in the evaluated `PolicyEngine`;
4. invokes an extended *release-gate* suite of staged workloads (e.g., frequency controls, emergence guardrails, credential-phishing scenarios).

The driver emits a machine-readable summary (exit codes and version-control metadata) suitable for archival with the manuscript.

Scope. The procedure yields *harness-level* evidence pertinent to Section 10; it does not replace field studies with statistical representativeness claims.

B Controlled ablation protocol

Driving script. `scripts/run_ablation_eval.py` executes three pytest subprocesses with `AGENTIUM_EVALUATION_ABLATION=1` and variant `AGENTIUM_ABLATION_VARIANT` set to `full`, `no_manifest`, or `permissive` over the tagged paper subset, verifies that failing node identifiers match `configs/ablation/paper_scenarios.json`, emits JUnit XML plus `ablation_summary.json` under timestamped directories beneath `artifacts/`, and optionally records classifier overhead via `scripts/ablation_microbench.py`. Production shells keep `AGENTIUM_EVALUATION_ABLATION` unset.

Scenario artefact. The JSON map lists pytest node identifiers for hypotheses H1–H6 alongside the expected degraded-mode failure unions; archival evidence templates live in `docs/evidence/`.

Table 4: Correspondence between hypotheses H1–H6 (Section 9) and verification roles. The *paper replication subset* denotes nine automated tests; the *release-gate suite* supplements them with broader stressors.

H	Acceptance criterion (summary)	Paper replication subset	Release-gate suite
H1	Undeclared tools fail closed; manifest-aligned ingress; auditable denial	Registry allowlist and HTTP ingress tests	Governance; privilege-escalation workloads
H2	High-risk path requires approval; no bypass of human gate	Approval workflow tests (three cases)	Governance workloads
H3	Background triggers guarded; bounded degrade and recovery	Background guard integration test	Emergence and outreach workloads
H4	Outbound DLP blocks synthetic secret-shaped payloads in the tagged harness test (repository revision pinned with harness outputs)	DLP pipeline test	Security; phishing workloads
H5	Cross-tenant access denied explicitly	Memory tenancy unit test	Governance workloads
H6	Reliability drills report MTTR and safe-degrade	Drill-runner unit test	Reliability workloads

Microbenchmark. `scripts/ablation_microbench.py` benchmarks *control-plane overhead only*: it invokes a trivial registered tool (`echo_micro`) many times after warmup (defaults: `-warmup 40`, `-iterations 400`) and compares latency when `ToolRegistry` is built without `dlp_classifier` versus with `DLPClassifier()` attached—both otherwise share the same allow rule and budgets. Emitted JSON (`microbench.json` when run standalone, or the same payload under `microbench` inside `ablation_summary.json` after `run_ablation_eval.py`, unless `-skip-microbench`) reports `p50_ms`, `p95_ms`, and `mean_ms` per stack (`minimal_stack`, `with_dlp_stack`); field `scenario` is fixed to `echo_tool_execution_overhead`. The measurement isolates benign echo-shaped traffic (*no* LLM calls) and therefore does *not* bound production tail latency, adversarial inputs, or end-to-end request time; see repository `docs/REPRO.md` for commands.